

An Ascon-AEAD128 Hardware Accelerator

Model-driven, open-source, validated on silicon

Mohammadi Masoudi Ramin

Politecnico di Milano

NEORV32 RISC-V SoC · Tang Nano 20K · fully-open toolchain

Outline

- ① Motivation
- ② Architecture
- ③ Verification
- ④ Challenges
- ⑤ Results
- ⑥ Wrap-up

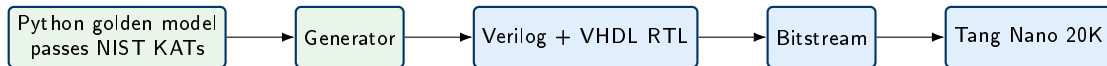
Why this project

- **Ascon** is the NIST lightweight-crypto standard (SP 800-232) for authenticated encryption.
- On a small CPU, computing it in software is slow — lots of 64-bit work on a 32-bit core.
- **Goal:** a hardware accelerator that does Ascon-AEAD128, usable from C, built with *only open tools*, and *proven on real silicon*.

Four objectives

- ① Hardware Ascon, matching the standard
- ② Clean C interface on RISC-V
- ③ Open, reproducible toolchain
- ④ Validate on FPGA + measure speed-up

From specification to silicon



Model-first design

The RTL is *generated* from a validated Python model, which also serves as the checking oracle. RTL and reference share one definition of “correct” — so bugs are interface bugs, never crypto bugs.

The accelerator: five verified layers

`perm_iter_r1` — permutation, 1 round/cycle

`aead128_core` — AEAD mode datapath

`aead128_axis` — AXI-Stream engine (FSM)

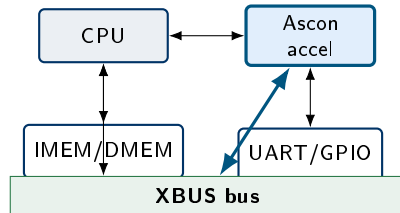
`aead128_axis_mmio` — register file

`aead128_wb` — Wishbone / XBUS adapter

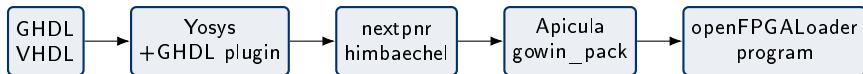
Each layer has one job and is checked against the model in isolation.

Integration on the NEORV32 SoC

- CPU: NEORV32, rv32i, 27 MHz.
- Accelerator is a **bus peripheral** on the external bus (XBUS) at 0x9000 0000
 - no CPU pipeline changes.
- Driver `ascon_accel.h` hides all registers behind `encrypt()` / `decrypt()`.



Fully-open, reproducible toolchain



No vendor EDA

Pinned by a Nix flake for exact reproducibility. Every step from VHDL/Verilog to programmed FPGA is open source.

Four layers of verification

Layer	Checks	Against
1	Python golden model	NIST Known-Answer Tests
2	Generated RTL	the model (25 vectors/module)
3	C driver	an emulated peripheral
4	End-to-end co-sim	NEORV32-faithful XBUS master

119 automated tests

Run from a clean checkout; reproduced by `nix flake check`. Layer 4 replays the driver's real register order through the bus adapter with NEORV32's exact pulsed-strobe timing.

From “simulation passes” to “correct on silicon”

Eight real bugs — each diagnosed from concrete evidence, not guesswork:

- ① Read-only build environment
- ② Toolchain name mismatch
- ③ Float-ABI (double vs. soft) mismatch
- ④ **Stack past end of RAM** → silent trap
- ⑤ Flaky USB debugger / shifting ports
- ⑥ **XBUS bus handshake**
- ⑦ FPGA fit (drop M extension)
- ⑧ **START-first protocol order**

The two that mattered

A processor panic with a precise fault address (0x9000 0000, 0x9000 0044) named exactly which register — and mechanism — had failed.

The key hardware bug: the bus handshake

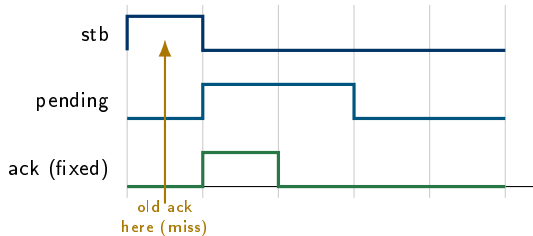
NEORV32's XBUS:

- pulses the strobe for *one cycle*
- samples the acknowledge only in the *next* cycle

First adapter: acknowledged *during* the strobe — the one cycle the CPU wasn't looking. $\Rightarrow$ CPU

never sees ack $\rightarrow$ bus timeout $\rightarrow$ **access fault**. **Fix:**

a handshake FSM that latches the strobe and acks the following cycle; re-verified against a NEORV32-faithful master.

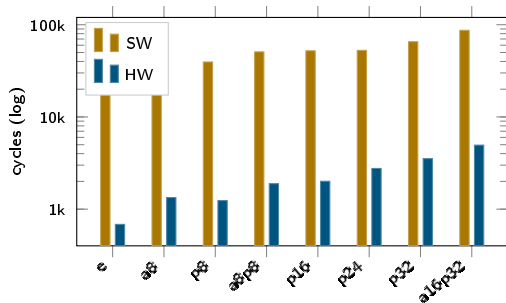


Results: correct, and 18–57× faster

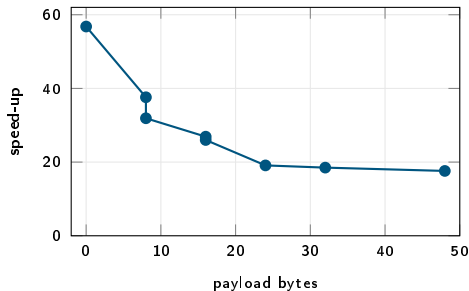
All 8 cases correct on hardware (enc_ok, dec_ok, tag_valid all set). Cycles at 27 MHz:

Case	AD	PT	SW	HW	Speed-up
empty	0	0	38 866	684	56.8 ×
ad8	8	0	50 205	1 335	37.6×
pt8	0	8	39 458	1 239	31.9×
ad8_pt8	8	8	50 797	1 890	26.9×
pt16	0	16	52 156	2 007	26.0×
pt24	0	24	52 748	2 763	19.1×
pt32	0	32	65 447	3 531	18.5×
ad16_pt32	16	32	86 984	4 948	17.6×

Results, visualised



cycles: software vs. hardware



speed-up falls as payload grows

Larger payloads $\Rightarrow$ more word-at-a-time data movement over MMIO $\Rightarrow$ lower speed-up. The crypto itself is a handful of cycles.

Faster than *what?* — being honest

The comparison

Software = CPU cycles (`rdcycle`) for a reference C Ascon-AEAD128 on the same rv32i core. **Hardware** = the accelerator's busy-cycle counter, same 27 MHz.

Caveats (stated plainly):

- Software is a *reference*, not hand-optimised — an optimised version would narrow the gap (hardware still wins).
- The hardware counter excludes register setup + readback; a full end-to-end wall-clock number is lower, especially for the tiny empty case.
- The $\sim 18\times$ on real payloads is closer to end-to-end than the $\sim 57\times$ on the empty one.

Fair phrasing: **the hardware engine vs. reference software on the same SoC** — same platform, same clock.

Limitations & future work

Limitations

- 32-byte payload cap
- MMIO word-at-a-time = data-movement bottleneck
- AEAD128 only on-board
- No side-channel analysis

Future work

- Rate-block streaming (arbitrary length)
- DMA / wider datapath
- Hash & XOF modes on-chip
- End-to-end wall-clock metric

Conclusion

What was achieved

A standard-conformant Ascon-AEAD128 accelerator — generated from a verified model, usable from C on a RISC-V SoC, built with *only open tools*, and **validated on silicon**: all 8 cases correct, **18–57** × faster than reference software on the same core.

- The hard part was the **hardware/software boundary** — the bus handshake and the protocol order — exactly what only real, end-to-end validation exposes.
- Every bug was found from the board's own evidence.

Thank you.

Ascon-AEAD128 · NEORV32 · Tang Nano 20K · fully-open